

HDOC Day-29 — Array Operations Using Menu-Driven Program

Algorithm

1. Start.
2. Input the size n of the array.
3. Input n array elements.
4. Display the menu:
 - Bubble Sort and Binary Search
 - Insertion and Deletion
 - Right Rotation by k positions
 - Exit
5. Read the user's choice.
6. Use `switch-case`:
 - **Choice 1:** Sort the array in ascending order using Bubble Sort. Input the element to search and perform Binary Search.
 - **Choice 2:** Ask whether the user wants insertion or deletion. For insertion, input the element and valid position and shift elements right. For deletion, input a valid position and shift remaining elements left.
 - **Choice 3:** Input k and rotate the array to the right by k positions.
 - **Choice 4:** Exit.
 - Otherwise, display "Invalid choice".
7. Repeat the menu until Choice 4 is selected.
8. Stop.

Test Case Table

Test Case	Operation	Input	Expected Output	Result
1	Bubble Sort + Binary Search	Array: 5 2 8 1 4, Search: 4	Sorted: 1 2 4 5 8; 4 found	Pass
2	Insertion	Array: 10 20 30 40, Insert 25 at position 3	10 20 25 30 40	Pass
3	Deletion	Array: 10 20 30 40 50, Delete position 3	10 20 40 50	Pass
4	Right Rotation	Array: 1 2 3 4 5, $k = 2$	4 5 1 2 3	Pass
5	Binary Search – absent	Array: 8 3 1 6 4, Search: 7	Sorted: 1 3 4 6 8; 7 not found	Pass
6	Invalid menu choice	Choice = 6	Invalid choice!	Pass

C Program

```

#include <stdio.h>

int main()
{
    int a[100];
    int n, choice, subChoice;
    int i, j, temp;
    int key, low, high, mid, found;
    int position, element;
    int k;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion\n");
        printf("3. Right Rotation by k positions\n");
        printf("4. Exit\n");
        printf("===== \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                /* Bubble Sort */
                for (i = 0; i < n - 1; i++)
                {
                    for (j = 0; j < n - i - 1; j++)
                    {
                        if (a[j] > a[j + 1])
                        {
                            temp = a[j];
                            a[j] = a[j + 1];
                            a[j + 1] = temp;
                        }
                    }
                }

                printf("\nSorted array: ");
                for (i = 0; i < n; i++)
                {
                    printf("%d ", a[i]);
                }

                /* Binary Search */
                printf("\nEnter element to search: ");
                scanf("%d", &key);
            }
        }
    }

```

```

low = 0;
high = n - 1;
found = 0;

while (low <= high)
{
    mid = (low + high) / 2;

    if (a[mid] == key)
    {
        found = 1;
        break;
    }
    else if (key < a[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}

if (found)
{
    printf("%d found at position %d.\n",
           key, mid + 1);
}
else
{
    printf("%d not found in the array.\n", key);
}

break;

case 2:
printf("\n1. Insertion\n");
printf("2. Deletion\n");
printf("Enter your choice: ");
scanf("%d", &subChoice);

switch (subChoice)
{
    case 1:
        if (n >= 100)
        {
            printf("Array is full.\n");
            break;
        }

        printf("Enter element to insert: ");
        scanf("%d", &element);

        printf("Enter position (1 to %d): ", n + 1);
        scanf("%d", &position);

```

```

        if (position < 1 || position > n + 1)
        {
            printf("Invalid position!\n");
        }
        else
        {
            for (i = n; i >= position; i--)
            {
                a[i] = a[i - 1];
            }

            a[position - 1] = element;
            n++;

            printf("Array after insertion: ");
            for (i = 0; i < n; i++)
            {
                printf("%d ", a[i]);
            }
            printf("\n");
        }
        break;

case 2:
    if (n == 0)
    {
        printf("Array is empty.\n");
        break;
    }

    printf("Enter position to delete (1 to %d): ", n);
    scanf("%d", &position);

    if (position < 1 || position > n)
    {
        printf("Invalid position!\n");
    }
    else
    {
        element = a[position - 1];

        for (i = position - 1; i < n - 1; i++)
        {
            a[i] = a[i + 1];
        }

        n--;

        printf("Deleted element: %d\n", element);

        printf("Array after deletion: ");
        for (i = 0; i < n; i++)
        {
            printf("%d ", a[i]);
        }
        printf("\n");
    }
}

```

```

        break;

        default:
            printf("Invalid choice!\n");
    }

    break;

case 3:
    if (n == 0)
    {
        printf("Array is empty.\n");
        break;
    }

    printf("Enter number of positions to rotate (k): ");
    scanf("%d", &k);

    k = k % n;

    for (j = 0; j < k; j++)
    {
        temp = a[n - 1];

        for (i = n - 1; i > 0; i--)
        {
            a[i] = a[i - 1];
        }

        a[0] = temp;
    }

    printf("Array after right rotation: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");

    break;

case 4:
    printf("Exiting program...\n");
    break;

default:
    printf("Invalid choice! Please enter 1 to 4.\n");
}

} while (choice != 4);

return 0;
}

```

Understanding Bubble Sort

For example, if the array is:

5 2 8 1 4

Bubble Sort repeatedly compares **two adjacent elements** and swaps them when the left element is greater than the right element. After sorting:

1 2 4 5 8

Test Case Evaluation

Test Case 1: The program correctly sorts the array using Bubble Sort and successfully finds 4 using Binary Search. **Result: Pass**

```
(kali㉿kali)-[~/Desktop]
$ cc day29.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter number of elements: 5
Enter 5 elements:
5 2 8 1 4

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
=====
Enter your choice: 1

Sorted array: 1 2 4 5 8
Enter element to search: 4
4 found at position 3.
```

Test Case 2: The program correctly inserts 25 at position 3 and shifts the remaining elements to the right. **Result: Pass**

```
Enter number of elements: 4
Enter 4 elements:
10 20 30 40

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
=====
Enter your choice: 2

1. Insertion
2. Deletion
Enter your choice: 1
Enter element to insert: 25
Enter position (1 to 5): 3
Array after insertion: 10 20 25 30 40
```

Test Case 3: The program correctly deletes the element at position 3 (30) and shifts the remaining elements to the left. **Result: Pass**

```

Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
=====

Enter your choice: 2

1. Insertion
2. Deletion
Enter your choice: 2
Enter position to delete (1 to 5): 3
Deleted element: 30
Array after deletion: 10 20 40 50

```

Test Case 4: The program correctly rotates the array two positions to the right. **Result: Pass**

```

Enter number of elements: 5
Enter 5 elements:
1 2 3 4 5

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
=====

Enter your choice: 3
Enter number of positions to rotate (k): 2
Array after right rotation: 4 5 1 2 3

```

Test Case 5: The array is first sorted, and Binary Search correctly determines that 7 is absent. **Result: Pass**

```

Enter number of elements: 5
Enter 5 elements:
8 3 1 6 4

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
=====

Enter your choice: 1

Sorted array: 1 3 4 6 8
Enter element to search: 7
7 not found in the array.

```

Test Case 6: The program correctly detects an invalid menu choice and displays the appropriate error message. **Result: Pass**

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
=====
Enter your choice: 6
Invalid choice! Please enter 1 to 4.
```

Conclusion

All **6 test cases passed successfully**. Therefore, the program correctly performs **Bubble Sort, Binary Search, insertion, deletion, right rotation, and menu-choice validation** as required in HDOC Day-29.